

EE 419 Project 3 Design Document

Geeoon Chung and Nate Snyder

Overall System

Our file serving system is loosely based on the TCP protocol. Due to the high packet loss we could face, TCP is a reasonable choice.

Connection Starting

We follow a similar protocol to TCP's 3-way handshake.

1. Client sends SYN packet with the file name.
2. Host sends a SYNACK packet with the sequence number of the final DATA packet.
3. Client sends an ACK packet with the value of 1.
4. The connection is now established.

Data Transfer

In order to simplify the protocol, sequence numbers are tied to the actual packet itself, not the number of bytes sent over.

1. Server sends a DATA packet with the expected sequence number (the last received ACK number).
 - a. This is done in a window, so it simultaneously sends the next $\text{WINDOW_SIZE} - 1$ packets as well.
2. Client sends an ACK with the last received DATA packet's sequence number + 1

Lost Packets

Client Timeout

Once the client sends an ACK, a timer starts. If the client does not receive any DATA packets, it will resend the ACK.

Server Timeout

Once the server sends a DATA packet, a timer starts. If the server does not receive any ACKs for that packet, the window will be set back to that DATA packet and all packets will be resent.

Dropped Packet

If the client receives a DATA packet that is ahead of what it is expecting (i.e., an intermediate DATA packet or ACK was dropped), then the client will send the last ACK. The sequence number on this ACK will be the sequence number + 1 for the last in-sequence DATA packet. The server will recognize that it's receiving either the same or previous received sequence number and roll back accordingly. It will then resend all packets in the window

Connection Teardown

Because the final sequence number was determined during the handshake, the server will automatically close the connection once it receives the final ACK. If it does not receive the final ACK, it will retry 3 times until it is assumed that the client's final ACK was dropped. On the client's end, once it sends the final ACK, it will wait an extra amount of time for any possible DATA packets before closing.

Final Protocol Design

Packet Format

Overall

Every packet sent has a one byte header. The following headers are possible: *S*, *B*, *A*, and *D*. These correspond to *SYN*, *SYNACK*, *ACK*, and *DATA*.

SYN

The SYN packet is used for establishing the connection. It is sent from the requesting “client” to the hosting “server.” It has the following format

Byte 1: *S*

Bytes 2-end: File name

SYNACK

The SYNACK packet is used for acknowledgement of the SYN packets, and for sending the number of packets to be sent. It is sent from the server to the client. It has the following format:

Byte 1: *B*

Bytes 2-5: Big endian integer representing the number of DATA packets to be sent.

ACK

The ACK packet is used for acknowledgement of DATA packets and for the acknowledgement of the SYNACK, indicating the connection has been established. It includes the next DATA packet that the server should send. It is sent from the client to the server. It has the following format:

Byte 1: *A*

Bytes 2-5: Next DATA packet sequence number

DATA

The DATA packet is used for sending data over the connection. It includes the sequence number of the packet as well as the data itself. It has a maximum size of 10205 (including the header). It is sent from the server to the client. It has the following format:

Byte 1: *D*

Bytes 2-5: Sequence number

Bytes 6-end: Binary data

State Transition Diagram